```
import discord
from discord.ext import commands
import random
import string
import asyncio
import os
from flask import Flask
import threading
import aiohttp

TOKEN = os.getenv('DISCORD_BOT_TOKEN')
WEBHOOK_URL = os.getenv('WEBHOOK_URL')

if not TOKEN:
    exit(1)

# ====================== FLASK ======================
app = Flask(__name__)

@app.route('/')
def home():
    return 'Bot en ligne !'

def run_flask():
    port = int(os.environ.get('PORT', 8080))
    app.run(host='0.0.0.0', port=port)

# ====================== BOT ======================
intents = discord.Intents.default()
intents.message_content = True

bot = commands.Bot(command_prefix='!', intents=intents)

is_searching = False
CHECKER_API = 'https://api.pomelo.lixqa.cc/v1/lookups'

CONCURRENT_TASKS = 5
DELAY_BETWEEN_BATCHES = 1.5
RATE_LIMIT_PAUSE = 10

def generate_4char_letters():
    return ''.join(random.choices(string.ascii_lowercase, k=4))

async def send_webhook(session, username):
```

```
    if not WEBHOOK_URL:
        return
    embed = {
        'title': 'Pseudo disponible !',
        'description': '@' + username,
        'color': 0x00ff00
    }
    try:
        await session.post(WEBHOOK_URL, json={'embeds': [embed]})
    except Exception:
        pass

async def check_username_api(session, username):
    try:
        async with session.post(
            CHECKER_API,
            json={'username': username},
            timeout=aiohttp.ClientTimeout(total=10)
        ) as r:
            if r.status == 429:
                return False, True
            if r.status == 200:
                data = await r.json()
                available = data.get('available') is True or data.get('status') =
                return available, False
            return False, False
    except Exception:
        return False, False

@bot.event
async def on_ready():
    print('Bot connecte : ' + str(bot.user))

@bot.command()
async def find4inf(ctx):
    global is_searching
    if is_searching:
        await ctx.send('Une recherche est deja en cours !')
        return

    is_searching = True
    await ctx.send('Recherche lancee ! Tape !stop pour arreter.')

    found = 0
    tested = 0
    last_report = 0
```

```
    async with aiohttp.ClientSession() as session:
        try:
            while is_searching:
                usernames = [generate_4char_letters() for _ in range(CONCURRENT_T

                results = await asyncio.gather(
                    *[check_username_api(session, u) for u in usernames]
                )

                rate_limited = any(rl for _, rl in results)

                if rate_limited:
                    await ctx.send('Rate limit detecte ! Pause de ' + str(RATE_LI
                    await asyncio.sleep(RATE_LIMIT_PAUSE)
                    continue

                tested += CONCURRENT_TASKS

                for username, (available, _) in zip(usernames, results):
                    if available:
                        found += 1
                        await ctx.send('DISPONIBLE ! @' + username)
                        await send_webhook(session, username)

                if tested // 100 > last_report // 100:
                    last_report = tested
                    await ctx.send(
                        'Rapport - ' + str(tested) + ' tentatives\n'
                        + 'Disponibles : ' + str(found) + '\n'
                        + 'Indisponibles : ' + str(tested - found)
                    )

                await asyncio.sleep(DELAY_BETWEEN_BATCHES)

        except asyncio.CancelledError:
            pass
        finally:
            is_searching = False
            await ctx.send(
                'Recherche arretee.\n'
                + 'Tentatives : ' + str(tested) + '\n'
                + 'Disponibles : ' + str(found)
            )

@bot.command()
async def stop(ctx):
    global is_searching
```

```
    if is_searching:
        is_searching = False
        await ctx.send('Arret en cours...')
    else:
        await ctx.send('Aucune recherche en cours.')

# ====================== LANCEMENT ======================
if __name__ == '__main__':
    threading.Thread(target=run_flask, daemon=True).start()
    bot.run(TOKEN)
```